



UNICAMP



Aula 6: Skills, Planejamento e Trace

Capacidades sob demanda, plano explícito e trace

Marcelo S. Reis [†]

[†] Departamento de Sistemas de Informação
Instituto de Computação, Universidade Estadual de Campinas

Campinas, 10 de setembro de 2026

Sumário

Skills

Planejamento

Observabilidade

Avaliar e entregar

Objetivos da aula

- ❖ Organizar instruções e ferramentas em capacidades carregadas sob demanda
- ❖ Distinguir plano implícito de plano explícito
- ❖ Decidir quando planejar antes de agir compensa
- ❖ Registrar a execução de forma que permita depurar e atribuir custo
- ❖ Fechar os requisitos do Entregável 3

O prompt que não para de crescer

Na Aula 5 separamos responsabilidades em agentes. Dentro de cada agente, o problema volta em escala menor:

- ❖ instruções para casos raros ocupam espaço em toda chamada
- ❖ ferramentas pouco usadas continuam visíveis
- ❖ o modelo lê tudo para usar quase nada

Contexto é recurso escasso, e a conta chega em custo e em precisão

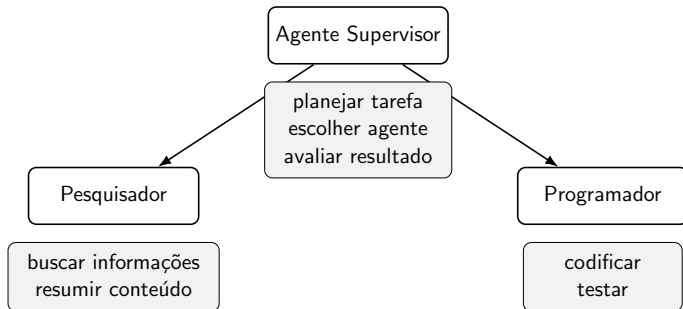
O que é uma skill

Uma capacidade nomeada, com três partes:

- ❖ instrução de como executar aquela tarefa
- ❖ ferramentas de que ela depende
- ❖ critério do que conta como sucesso

A diferença em relação a escrever tudo no prompt está no **momento do carregamento**: a instrução completa entra no contexto quando a skill é escolhida (buscando por índices), não antes

Exemplo de skills em sistemas multiagentes



Agentes definem **quem faz**; skills definem **o que sabem fazer**;
ferramentas definem **com quais recursos fazem**

Divulgação progressiva

Sempre no contexto	nome e uma linha sobre quando usar cada skill
Sob demanda	instrução completa, exemplos, formato de saída
Nunca	material de referência que a ferramenta pode buscar

O índice precisa ser bom o bastante para a escolha ser correta com pouca informação

Skill, ferramenta e agente

	O que é	Sinal de escolha
Ferramenta	uma ação executável	há efeito ou dado fora do modelo
Skill	instrução mais ferramentas para uma tarefa	o <i>como</i> é longo e específico
Agente	escopo próprio, contexto próprio, avaliação própria	precisa decidir sozinho e ser testado à parte

Muita coisa que vira agente deveria ter sido skill

Plano implícito e plano explícito

**Implícito
(ReAct)**

o próximo passo emerge da observação anterior

Explícito

o sistema escreve a sequência antes de executar

No plano explícito, a sequência vira um artefato: pode ser lida, avaliada, corrigida e comparada com o que de fato aconteceu

Quando planejar antes compensa

- ❖ A tarefa tem várias partes independentes
- ❖ Errar a ordem custa caro
- ❖ Passos podem ser executados em paralelo
- ❖ O usuário precisa aprovar antes da execução

E quando atrapalha:

- ❖ tarefas curtas, em que o plano custa mais que a execução
- ❖ ambientes em que cada observação muda o que faz sentido fazer a seguir

Replanejar

Um plano feito antes da primeira observação envelhece rápido

- ❖ replanejar a cada passo aproxima o sistema do ReAct, com custo maior
- ❖ nunca replanejar produz execução fiel a um plano errado
- ❖ o meio-termo comum é replanejar diante de falha ou de resultado inesperado

Registre plano inicial e plano final. A diferença entre os dois costuma ser o achado mais interessante da análise

Planejamento no estudo de caso

“Quem pode participar, quais documentos preciso e até quando envio?”

Plano possível:

1. consultar elegibilidade
2. consultar documentos obrigatórios
3. consultar prazo
4. sintetizar com evidências

Os três primeiros passos são independentes. Isso é uma escolha de projeto observável nos números, não um detalhe de implementação

O que é um trace

O registro ordenado do que o sistema fez, e não apenas do que respondeu

- ❖ quem foi acionado, em que ordem
- ❖ que entrada cada agente recebeu
- ❖ que ferramentas foram chamadas, com quais argumentos
- ❖ quanto custou cada etapa

Três perguntas que o trace precisa responder

1. Por que o sistema tomou esse caminho?
2. Onde a informação correta se perdeu?
3. Qual etapa dominou custo e latência?

Se o registro atual não responde às três, ele ainda não é observabilidade: é log

Atribuir custo

Por agente

mostra quem domina o custo

Por etapa do plano

mostra se a decomposição valeu a pena

Por caso de teste

mostra se o custo se concentra em poucas entradas

- ❖ Um agente que responde por 70% da latência é candidato natural a modelo menor, cache ou remoção

Depurar um sistema multiagente

Procedimento que funciona:

1. confirme que a informação existia na fonte
2. verifique se a ferramenta a devolveu
3. verifique se o especialista a incluiu no achado
4. verifique se a síntese a preservou

O ponto onde a cadeia quebra determina o que corrigir. Sem trace, o diagnóstico vira tentativa e erro sobre o(s) prompt(s)

Avaliação fim a fim e por parte

A régua principal continua sendo o conjunto congelado, aplicado ao sistema inteiro (fim a fim)

Medidas por parte (internas) servem para diagnosticar:

- ❖ o supervisor escolheu o especialista certo?
- ❖ o especialista respondeu dentro do escopo?
- ❖ a síntese preservou as evidências recebidas?

Melhorar apenas uma medida interna também é um resultado, e vale relatar

Entregável 3: o que será avaliado

- ❖ Justificativa da divisão, a partir de limitações observadas na v2
- ❖ Pelo menos dois agentes com responsabilidades distintas
- ❖ Contrato explícito entre eles
- ❖ Skills ou planejamento, quando o problema pedir
- ❖ Trace que permita depurar e atribuir custo
- ❖ Comparação v2 contra v3 na mesma régua

Prazo: **20 de setembro**. Enunciado e template serão disponibilizados no Google Classroom

Atividade prática

No notebook da Aula 6:

1. damos ao supervisor um plano explícito
2. carregamos instrução por skill, sob demanda
3. registramos o trace completo da execução
4. atribuímos custo e latência por agente
5. usamos o trace para explicar uma falha

Ponte para o Entregável 4

Com o sistema dividido, fragilidades ficam visíveis:

- ❖ falhas de ferramenta e de rede em qualquer etapa
- ❖ variação entre execuções idênticas
- ❖ conclusões tiradas de conjuntos pequenos
- ❖ diluição de responsabilidades (com possíveis implicações éticas)

Tratamento de erros, confiabilidade, avaliação ética e comparação arquitetural final.